

Anda: Unlocking Efficient LLM Inference with a Variable-Length Grouped Activation Data Format

Chao Fang^{†,‡}, Man Shi^{†,*}, Robin Geens[‡], Arne Symons[‡], Zhongfeng Wang^{†,*}, Marian Verhelst[‡]

[†]School of Electronic Science and Engineering, Nanjing University, China [‡]ESAT-MICAS, KU Leuven, Belgium

Email: fantasysee@smail.nju.edu.cn, zfwang@nju.edu.cn, {man.shi, robin.geens, arne.symons, marian.verhelst}@kuleuven.be

Abstract—The widely-used, weight-only quantized large language models (LLMs), which leverage low-bit integer (INT) weights and retain floating-point (FP) activations, reduce storage requirements while maintaining accuracy. However, this shifts the energy and latency bottlenecks towards the FP activations that are associated with costly memory accesses and computations. Existing LLM accelerators focus primarily on computation optimizations, overlooking the potential of jointly optimizing FP computations and data movement, particularly for the dominant FP-INT GeMM operations in LLM inference.

To address these challenges, we investigate the sensitivity of activation precision across various LLM modules and its impact on overall model accuracy. Based on our findings, we first propose the Anda data type: an adaptive data format with group-shared exponent bits and dynamic mantissa bit allocation. Secondly, we develop an iterative post-training adaptive precision search algorithm that optimizes the bit-width for different LLM modules to balance model accuracy, energy efficiency, and inference speed. Lastly, a suite of hardware optimization techniques is proposed to maximally exploit the benefits of the Anda format. These include a bit-plane-based data organization scheme, Anda-enhanced processing units with bit-serial computation, and a runtime bit-plane Anda compressor to simultaneously optimize storage, computation, and memory footprints. Our evaluations on FP-INT GeMM operations show that Anda achieves a $2.4\times$ speedup, $4.0\times$ area efficiency, and $3.1\times$ energy efficiency improvement on average for popular LLMs including OPT, LLaMA, and LLaMA-2 series over the GPU-like FP-FP baseline. Anda demonstrates strong adaptability across various application scenarios, accuracy requirements, and system performance, enabling efficient LLM inference across a wide range of deployment scenarios.

I. INTRODUCTION

Large language models (LLMs) [11], [63], [70], [72], [86] have demonstrated remarkable proficiency in a wide array of natural language processing tasks, including text generation, question answering, and automatic summarization. The extraordinary success of LLMs can be attributed to the scaling law [3], which posits that performance improves dramatically with the ever-increased model size, training data volume, and computational resources. The evolution of the GPT series [5], [38], [63] on model size strikingly illustrates the scaling law: while GPT-1 comprised a modest 117 million parameters, its successor GPT-4 is speculated to encompass over a trillion parameters. However, the exponential growth in LLM model sizes has created substantial deployment challenges, imposing enormous demands on storage and computational resources.

*Corresponding author

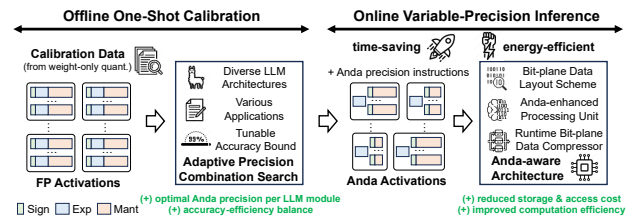


Fig. 1. Overview of the drop-in replacement for FP activations using the variable-length grouped Anda data type via a one-shot offline calibration process. This enables online variable-precision LLM inference, significantly improving speed and energy efficiency through the adaptive precision combination search algorithm and the Anda-aware architecture.

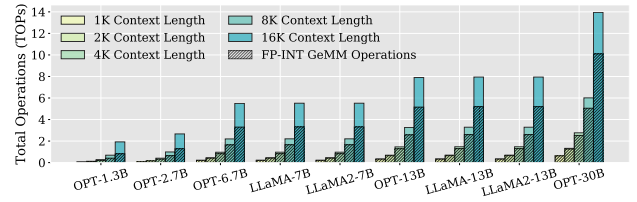


Fig. 2. Proportion of FP-INT GeMM operations in weight-only quantized LLMs across varying model sizes and context lengths for text generation tasks. FP-INT GeMMs dominate ($>90\%$) in prevalent sub-4K token applications and remain significant for 10K+ sequences.

To address these challenges, quantization techniques [12], [16], [24], [51], [52], [66], [78] have been widely adopted in LLMs to reduce memory footprint and lower deployment costs. To maximally shrink the model size, the most common strategy for LLMs today is weight-only quantization [8], [17], [24], [34], [35], [47], [51], [64], [66], [78], [81], which aggressively lowers the precision of the weights while maintaining high precision for activations due to the presence of outliers [16], [77]. In particular, the widely adopted W4A16 scheme [24], [66], which quantizes weights into 4-bit integers (INT4) while retaining activations in 16-bit floating-point format (FP16), significantly reduces memory requirements and bandwidth, lowering GPU memory usage by nearly $4\times$ [83] and facilitating deployment to smaller devices [51], [62].

With the increasing importance of weight-only quantization, FP-INT GeMM operations [32] have become indispensable in LLM inference. As illustrated in Fig. 2, FP-INT GeMMs constitute a significant portion of the computational workload across various weight-only quantized LLMs and context lengths in text generation tasks. They dominate in typical

applications with sequences under 4K tokens, comprising over 90% of operations on average, and remain substantial even for context lengths exceeding 10K tokens in applications from LongBench [4]. Such prevalence highlights the urgent need to optimize FP-INT GeMMs for efficient LLM inference.

NVIDIA’s new FP-INT GeMM kernel [62], as well as some specialized GPU kernels [24], [35], [57], [78], are a consequence of this evolution. However, these optimized GPU kernels still rely on using FP units by converting the INT weights to FP values for execution in FP GeMM operators [52]. Efforts have been made to develop dedicated FP-INT arithmetic units [32], while the additional costs of exponent alignment and normalization persist, resulting in complicated hardware implementation. To reduce hardware cost, another optimization approach is to convert the FP activations to a block floating point (BFP) data format for computation [13], [19], [44]. Since grouped BFP elements share an exponent, the overhead of exponent alignment and normalization within a group disappears, simplifying the operation to INT arithmetic. However, to mitigate accuracy loss when converting FP activations to BFP format on a pre-trained network, costly retraining [12]–[14], [26], [39], [44], [61], [85] is required, hindering agile LLM deployment. Alternatively, accuracy can be preserved by using a large mantissa field conversion [32], [42], but this significantly increases the energy consumption due to computational and memory access overhead of the additional bits.

In summary, processing FP activations remains a major bottleneck in weight-only quantized LLM inference, and existing methods struggle to balance model accuracy, computational efficiency, and energy consumption. To overcome the above limitations, we propose Anda to unlock efficient LLM inference. Anda introduces a novel variable-length grouped activation data format, coupled with the algorithm innovation and specialized hardware optimizations. As shown in Fig. 1, Anda first employs a fast, training-free adaptive precision search algorithm during compile time, using the same calibration data as post-training weight-only quantization [25]. Guided by user-defined accuracy constraints, our one-shot process identifies the desired mantissa bit length, which instructs the activation precision across various LLM modules during inference. Combining the flexible Anda data format with our specialized hardware architecture allows running these dominant FP-INT GeMM operations at lower precision, significantly improving inference speed and energy efficiency for weight-only quantized LLMs. More concretely, our contributions are as follows:

- We investigate the potential of BFP activation quantization across popular LLM models within different modules. Based on these insights, we propose Anda: a variable-length grouped data format with shared exponents and adjustable mantissa widths for activations.
- We develop an adaptive search algorithm to optimize the mantissa widths for different LLM modules without retraining. This algorithm balances model accuracy, energy efficiency, and inference speed based on a user-defined accuracy loss tolerance.

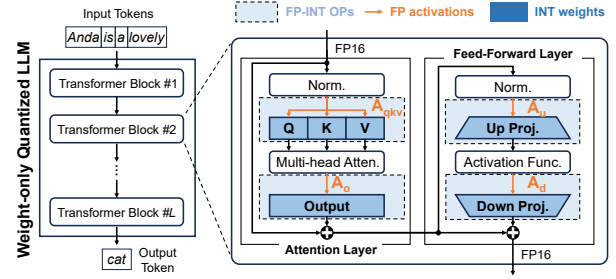


Fig. 3. Illustration of the architecture for a weight-only quantized LLM model.

- We design an efficient Anda-aware hardware architecture featuring (a) a bit-plane-based data layout scheme in memory, (b) Anda-enhanced bit-serial processing units, and (c) a runtime bit-plane data compressor. Extensive evaluations of the Anda system across popular LLMs demonstrate an average improvement in processing speed of $2.4\times$, area efficiency of $4.0\times$ and energy efficiency of $3.1\times$ compared to existing SotA hardware.

The paper is organized as follows: Sec. II reviews the benefits and remaining bottlenecks of weight-only quantized LLMs and BFP formats, and quantifies the sensitivity of LLM inference accuracy to the shared exponents and reduced mantissas sizes. Based on these findings, Sec. III features the proposed Anda data format and presents the algorithm to rapidly optimize mantissa length under given accuracy constraints. Next, an Anda-optimized hardware architecture is proposed in Sec. IV, which allows us to derive system-level gains in Sec. V and benchmark it against the SotA solutions. Sec. VII concludes the paper.

II. BACKGROUND AND MOTIVATION

A. Weight-only Quantized LLMs

Weight-only quantization [8], [17], [24], [34], [35], [47], [51], [64], [66], [78], [81] has emerged as a pivotal technique for efficient LLM inference. Unlike weight-activation quantization [12], [16], [52], [79], [89], which reduces precision for both weights and activations, weight-only quantization focuses solely on compressing model parameters using a much more aggressive quantization scheme.

Fig. 3 illustrates the architecture of a weight-only quantized LLM, composed of a series of Transformer blocks that each contains an attention layer and a feed-forward layer. The light blue background highlights the dominant computational modules involving FP-INT GeMM operations, which can be categorized into four module types based on the positions of the FP activations: the first type involves A_{qkv} interacting with W_q , W_k , and W_v to compute the query (Q), key (K), value (V) matrices, respectively; the second type involves A_o multiplying with W_o to compute the output matrix; the other two types are up-projection and down-projection modules of the feed-forward layer, respectively, involving A_u and A_d with interacting to corresponding weights.

Weight-only quantized LLMs offer significant advantages in storage efficiency [62], [66]. Compared to W8A8 weight-

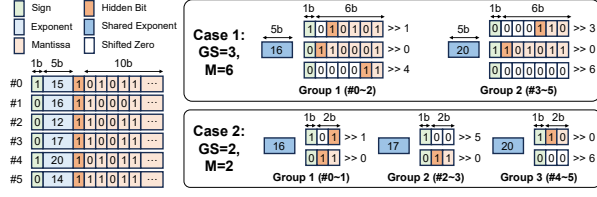


Fig. 4. The process of converting a set of FP16 numbers into different BFP numbers. BFP format is regulated by two key parameters: group size (GS) and mantissa length (M).

activation quantized LLMs [79], W4A16 weight-only quantized LLMs [51] achieve similar model accuracy while reducing storage requirements of model parameters by nearly half [83], making them particularly suitable for deployment on resource-constrained devices in edge computing scenarios. However, under current GPU computing schemes, computing a W4A16 FP-INT operations consumes approximately $1.7\times$ more energy than W8A8 INT-only operations [42]. This can be explained by accessing FP activations incurs higher energy costs than INT weights [31], and FP-INT operations require complicated hardware implementations [32]. Hence, optimizing FP activations emerges as a key opportunity to improve the overall efficiency of weight-only quantized LLMs.

B. Block Floating Point

Reducing the computation and storage overhead of FP16 activations is crucial for optimizing the efficiency of LLMs. BFP [19] offers a promising solution by sharing exponents within groups of values, preserving dynamic range while mitigating the impact of outliers and simplifying computations. The BFP format can be characterized by two key parameters: group size and mantissa length. Fig. 4 shows the process of converting FP16 tensors to BFP numbers using two different instances of the BFP format. Initially, FP16 tensors are divided into groups. Within each group, the largest exponent is selected as the shared exponent and other mantissas are right-shifted based on their exponent differences. Bits exceeding the specified mantissa length are truncated, and zero is represented by all mantissa bits being 0. As illustrated in Fig. 4, this conversion process can lead to precision loss due to mantissa truncation, with some elements becoming zero, thereby posing a significant challenge to maintaining model accuracy.

Current approaches to address this fall into two categories. On the one hand, BFP-aware training fine-tunes the model after the quantization [12]–[14], [23], [26], [39], [41], [44], [61], [85], at the expense of a costly training process, making it rather impractical for agile LLM deployment. On the other hand, direct conversion of pre-trained FP models to BFP formats [22], [23], [44], [50], [61] requires long mantissas to avoid the significant accuracy loss, which increases computation and storage overhead, diminishing the advantages of BFP. To avoid the storage of these long mantissas, methods like FIGNA [32] and [42] propose dynamic conversion to BFP during computation. This approach stores activations in FP16 format and expands to long mantissas with shared exponents

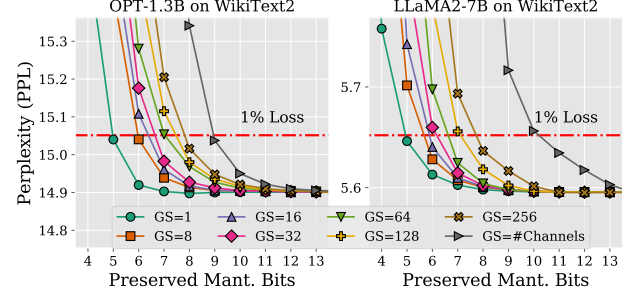


Fig. 5. LLM sensitivity to BFP group size (GS) and preserved mantissa bits.

before computations to maintain model accuracy. However, this also prevents FIGNA from obtaining activation memory footprint savings.

To avoid both costly retraining and large activation memory footprints, we seek a solution that can rapidly convert FP activations to BFP activations without retraining, while also leveraging the computational and storage advantages of BFP for LLM inference. To achieve this goal, it is necessary to explore opportunities for reduced mantissa length BFP under the unique characteristics of LLMs.

C. Opportunities towards Activation Optimizations

We explore opportunities for LLM activation optimization by investigating the sensitivity of model accuracy to reduced mantissa lengths in BFP formats. This study converts FP-INT GeMM activation tensors (A_{qkv} , A_o , A_u , A_d) from FP16 to BFP format, as shown in Fig. 4. Model accuracy is evaluated using perplexity (PPL) on the WikiText2 dataset, with lower PPL indicating higher accuracy. We assume a 1% accuracy loss tolerance in practical scenarios. We aim to uncover efficient activation representations while maintaining LLM performance within acceptable limits.

Sensitivity to group size: Fig. 5 illustrates the sensitivity to shared exponent group size for two different LLM models across various mantissa lengths. The experiments reveal a clear trade-off between group size and the minimum required mantissa length to maintain model accuracy. Larger activation group sizes allow more efficient parallel computations, yet at a greater accuracy tolerance or increased mantissa lengths. Based on these observations, we select a group size of 64 for subsequent experiments, as it offers a good balance between computational efficiency and accuracy tolerance.

Sensitivity to LLM model: With this group size of 64, we continue our exploration across a wider range of recent LLMs, to derive their sensitivity to reduced mantissa lengths. Fig. 6 reveals varying sensitivities among different models. Notably, models such as OPT-2.7B, OPT-6.7B, OPT-13B, and OPT-30B are less sensitive to mantissa reduction, allowing for the direct removal of 5 mantissa bits, while other models could only tolerate the removal of 4 mantissa bits. As more mantissa bits are removed, differences in accuracy sensitivity become more pronounced. This insight inspires us to consider a variable-length BFP datatype, potentially enabling more aggressive

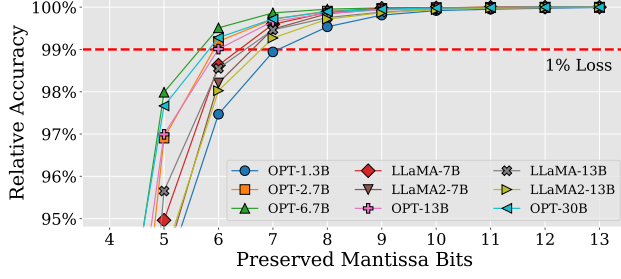


Fig. 6. The relative accuracy to preserved mantissa bits across various LLMs.

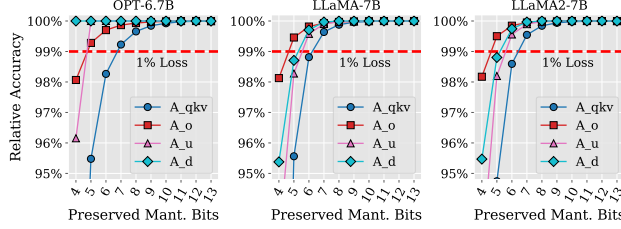


Fig. 7. The relative accuracy of OPT-6.7B, LLaMA-7B, and LLaMA2-7B when cutting mantissa bits on either A_{qkv} , A_o , A_u , or A_d activation only.

compression in less sensitive models while employing a more conservative one for others. It also prompts us to explore whether activations in different modules within one LLM have varying sensitivities.

Sensitivity to LLM inner module: We finally explore the impact of different mantissa lengths of the activations of different modules within the same LLM. More specifically, we examine the A_{qkv} , A_o , A_u , and A_d modules of the OPT-6.7B, LLaMA-7B, and LLaMA2-7B models. The mantissa length of each module is swept while keeping the lengths of other modules fixed at 13 bits. Fig. 7 summarizes the results, revealing that activations from different modules have varying impacts on model accuracy across all three models. A_{qkv} consistently shows the most significant influence, while A_d demonstrates low sensitivity in OPT-6.7B but has a more pronounced effect in the LLaMA series models.

Our study reveals several key insights into the application of BFP in LLMs: (a) LLMs can maintain good performance with reduced mantissa lengths. (b) Different LLM models exhibit varying sensitivities to mantissa reduction. (c) Within a single LLM, different modules have distinct sensitivities to precision reduction. These observations motivate us to introduce the new variable-length grouped data format for FP activations, along with a methodology for post-training quantization (PTQ) and rapid selection of tolerable reduced mantissa lengths for any LLM.

III. ANDA DATA FORMAT

In this section, we present unique features of the Anda data format and demonstrate its benefits towards FP-INT operations in weight-only quantized LLM inference. Furthermore, we introduce a mantissa bit-width search method to efficiently

TABLE I
ANDA FORMAT DEFINITION IN CONTRAST WITH PRIOR BFP FORMATS

BFP Type		Mantissa Length during Computation	Computation	Storage
VS-Quant [12]	Uni-Length	4b	Bit-parallel	BFP Element-based
BOOST [26]		5b		BFP Element-based
X. Lian et al. [50]		8b		BFP Element-based
FIGNA [32]		14b		FP16 Element-based
H. Fan et al. [22]		15b		BFP Element-based
Flexpoint [44]		16b		BFP Element-based
FAST [85]	Multi-Length	2b/4b	Chunk-serial	BFP Chunk-based
DaCapo [41]		2b/4b/8b	Bit-parallel	BFP Element-based
FlexBlock [61]		4b/8b/16b		BFP Element-based
Anda (Ours)	Variable-Length	1b/2b/.../16b	Bit-serial	BFP Bit-plane-based

identify the optimized Anda precision combinations that satisfy a user-defined accuracy drop.

A. Anda Format Features

Based on the findings of our previous study, we propose the Anda format: an innovative variable-length mantissa BFP scheme designed for efficient LLM inference. Anda's structure comprises a sign bit, a shared exponent, and a variable-length mantissa, building upon traditional BFP conversion processes as previously shown in Fig. 4. Its key feature is the ability to dynamically select mantissa lengths for different tensors based on their precision sensitivity, maintaining consistency within each tensor while optimizing the accuracy-efficiency trade-off.

Table I compares Anda with prior BFP formats, categorizing them based on supported mantissa lengths. Uni-length formats, such as VS-Quant [12] and FIGNA [32], use fixed mantissa lengths, while multi-length formats like FAST [85] and DaCapo [41] offer limited flexibility with 2~3 predefined lengths. Anda surpasses both by providing a continuous range of mantissa lengths, allowing fine-grained precision control across different LLM modules. Enabled by specialized hardware units, as detailed in Sec. IV, smaller mantissa widths result in a lower inference latency, computational cost and memory storage cost. This allows Anda format to carefully balance model precision and computational efficiency, providing a more aggressive compression in less sensitive model parts while preserving critical precision elsewhere.

B. Efficient FP-INT GeMM Using Anda Format

We then compare the workflows of GeMM workflows of several SotA approaches to illustrate the advantages of replacing FP16 activations with the Anda data format. Taking the W4A16 quantization scheme as an example, we examine the FP-INT GeMM computation process (a) on existing GPU platforms [52]; (b) on GPU platforms with dedicated FP-INT processing units; (c) using FIGNA's dynamic conversion scheme [32]; and (d) with our proposed Anda approach. Fig. 8 depicts the four schemes, with colors indicating the data types used throughout the computational process.

Fig. 8(a) shows the workflow of W4A16 LLMs on common GPU platforms. The absence of dedicated FP-INT computation units in GPU necessitates converting INT4 weights to FP16 before processing, with tensor cores operating in FP16 mode. This scheme not only brings additional format conversion overheads, but requires costly FP computations.

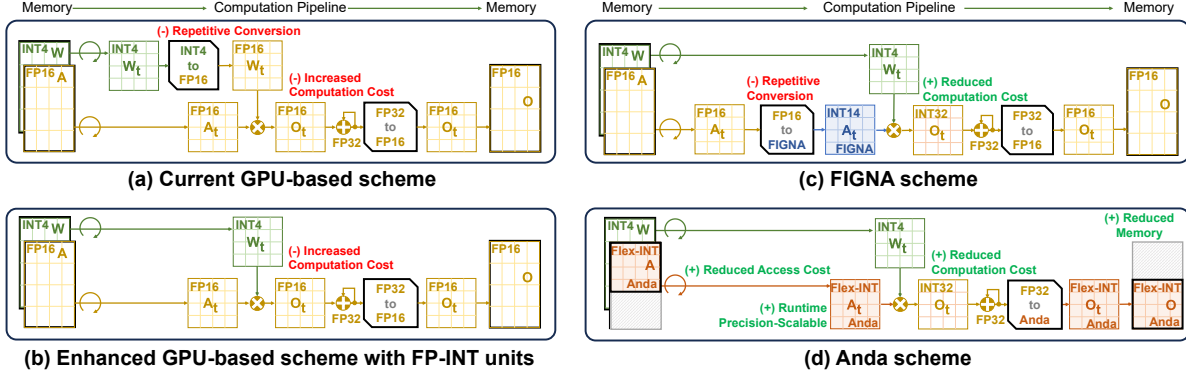


Fig. 8. Comparison of (a) the current computation scheme on GPU, (b) and that enhanced with dedicated FP-INT processing unit, (c) FIGNA scheme, and (d) our Anda scheme for FP-INT GeMM. Our Anda scheme significantly reduces memory space, access cost, and computation cost and enables energy-efficient precision-scalable operations.

GPU platforms equipped with dedicated FP-INT processing units, as illustrated in Fig. 8(b), can eliminate the need for converting INT4 weights to FP16, thereby reducing data conversion overheads and computation costs. However, as pointed out by FIGNA [32], the high alignment and normalization overhead associated with FP-INT processing units still results in high computational expenses.

To efficiently deploy W4A16 LLMs, FIGNA proposes a computation scheme using a BFP variant with corresponding hardware support to overcome the issues with dedicated FP-INT units. As depicted in Fig. 8(c), activations are stored in FP16 format in memory, converted to the FIGNA format before computation, after which a 14-bit mantissa is multiplied with INT4 weights for GeMM computation. The final results are then converted again to FP16 and written back to memory. This scheme reduces the computation overhead by converting costly FP GeMM to INT operations. However, since FP16 activations need to be repeatedly accessed during computation, frequent data conversion from FP16 to FIGNA introduces additional overhead, affecting overall efficiency.

As presented in Fig. 8(d), our proposed Anda format computation scheme offers some unique advantages in contrast with the previous approaches. Firstly, the activations are no longer stored in memory in FP16 format, but directly in the Anda data format, reducing storage overhead and data access overhead while avoiding frequent data conversion. Secondly, the shared exponent enables INT dot-product operations within a group, followed by FP32 accumulation across groups, reducing the computational overhead of FP-INT GeMMs. Thirdly, the variable-length mantissa considerably decreases dot-product operations and memory accesses use the minimal necessary word length. Finally, converting only the final FP32 results back to Anda format before writing to memory minimizes the storage requirement and the additional overhead from switching data format.

C. Adaptive Precision Combination Search

To leverage the Anda format for fast deployment and hardware performance gains, we propose an adaptive precision search algorithm for offline compile-time optimization

of activation precisions in weight-only quantized LLMs. Our algorithm is built around two key strategies. (a) We narrow the search space to the precision of only four key tensor types, i.e., A_{qkv} , A_o , A_u , and A_d , based on their sensitivity to model accuracy as demonstrated in Fig. 7. This precision combination is represented as a 4-tuple $[M_{qkv}, M_o, M_u, M_d]$. (b) We employ a training-free, one-shot calibration process reusing the small amount of calibration data from the post-training weight-only quantization process, being several thousands of tokens with hundred batches [24], [51], [66]. Though prior layer-wise methods [18], [28], [76] may achieve finer precision adjustments, their prolonged search times significantly extend the deployment process. In contrast, our module-wise approach rapidly assigns mantissa lengths while maintaining consistency across layers and can easily be integrated into standard post-training deployment workflows.

As outlined in Algorithm 1, we take the LLM model L , a calibration dataset D , an accuracy loss tolerance δ , and a maximum number of iterations N as inputs. The accuracy tolerance δ specifies the acceptable level of performance degradation, while the maximum number of iterations N serves as a termination criterion, ensuring the algorithm concludes within a reasonable time frame. With these inputs, our algorithm finds the optimal 4-tuple precision combination within the given iterations that best balances model accuracy and inference efficiency across the model's key activation components. The search process consists of three key steps.

Step 1: Initialize search starting points. A priority queue with precision combinations of equal precision across all modules is initialized first. These precision combinations range from aggressive (e.g., [4, 4, 4, 4]) to conservative (e.g., [13, 13, 13, 13]). This strategy enables the rapid discovery of efficient combinations while ensuring the existence of feasible solutions, as validated by our prior experiments in Fig. 6.

Step 2: Check the promising combination. In each iteration, the combination with the lowest bit operations (BOPs) is extracted from the priority queue and added to the visited

Algorithm 1: Adaptive Precision Combination Search

Input: LLM model L , calibration dataset D , accuracy loss tolerance δ , max iterations N
Output: Optimized precision combination $best_comb$ denoted as a 4-tuple $[M_{qkv}, M_o, M_u, M_d]$

// S1: Initialize search starting points
1: $Q \leftarrow \text{PriorityQueue}([4, 4, 4, 4], \dots, [13, 13, 13, 13]);$
2: $best_comb \leftarrow \text{null}, best_bops \leftarrow \infty;$
3: $iterations \leftarrow 0, visited \leftarrow \{\};$
4: $fp_acc \leftarrow \text{EvaluateAccuracy}(L, D);$
5: **while** $iterations < N$ **do**
 // S2: Check the promising combination
6: $bops_eval \leftarrow Q.\text{map}(\text{EvalBOPs});$
7: $curr_bops \leftarrow \min(bops_eval);$
8: $curr_comb \leftarrow Q.\text{get}(bops_eval.\text{index}(curr_bops));$
9: $visited \leftarrow visited \cup \{curr_comb\};$
10: $anda_acc \leftarrow \text{EvaluateAccuracy}(L, D, curr_comb);$
 // S3: Update and relax the best combination
11: **if** $curr_bops < best_bops$ **and** $anda_acc \geq (1 - \delta) \cdot fp_acc$ **then**
12: $best_comb \leftarrow curr_comb;$
13: $best_bops \leftarrow curr_bops;$
14: $neighbors \leftarrow \text{GenerateCandidates}(curr_comb);$
15: **foreach** $n \in neighbors$ **do**
16: **if** $n \notin visited$ **then**
17: $Q.\text{push}(n);$
18: **end**
19: **end**
20: **end**
21: **if** $Q.\text{empty}()$ **then**
22: **break;**
23: **end**
24: $iterations \leftarrow iterations + 1;$
25: **end**
26: **return** $best_comb$

set. The BOP metric [1], [43], [49], [71] quickly estimates computational cost by calculating the total number of bit operations for the necessary multiplications under a given combination. This allows us to efficiently prioritize promising combinations without a full model evaluation. The accuracy of the promising combination is then examined on the calibration dataset.

Step 3: Update and relax the best combination. If the evaluated combination yields lower BOPs than the current best while maintaining accuracy within the specified tolerance, it becomes the new best combination. To generate nearby precision candidates, the algorithm then relaxes this best combination by decreasing the mantissa length of each tensor type by one, while keeping the other tensor types unchanged. For example, if the current best combination is $[6, 7, 5, 5]$, the generated candidates will be $[5, 7, 5, 5]$, $[6, 6, 5, 5]$, $[6, 7, 4, 5]$, and $[6, 7, 5, 4]$. The generated candidates that have not been visited before are added to the priority queue. If the accuracy constraint is not met, no update is made. Step 2 and 3 are repeated until the maximum number of iterations is reached or the search space is exhausted.

D. Precision Combination Search Efficiency

Our algorithm aims to efficiently optimize FP activations in weight-only quantized LLMs during the post-training phase.

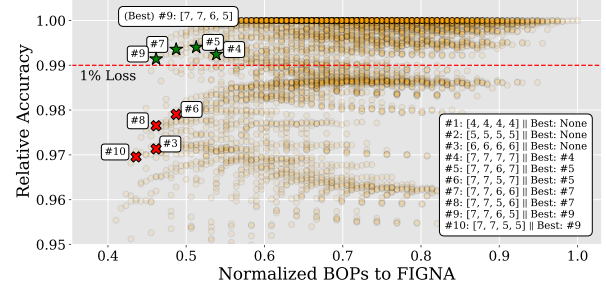


Fig. 9. Search process of the proposed adaptive precision combination search algorithm on the OPT-125M model with constraint under 1% accuracy loss, which efficiently finds the global optimum within 10 iterations.

Most weight-only quantization processes [24], [51], [66] rely on a small calibration dataset, which we can reuse in the activation precision search. Ensuring a rapid search process is critical to avoid extending post-training deployment time. Therefore, the algorithm is designed to find a near-optimal solution quickly, within an acceptable accuracy tolerance, to enable efficient hardware deployment.

The efficiency of our algorithm is enhanced by two key mechanisms: First, we introduce a constraint that updates the best combination only when a new precision combination offers a lower computational cost, employing a relaxation strategy similar to gradient descent to accelerate convergence. While this may miss the global optimum, it ensures a high-performance combination within limited iterations. Second, we set an iteration limit to complete the search within a reasonable timeframe, avoiding deployment delays. It is here important to note that the relatively limited search space of only 4 precision variables allows for fast convergence with just a few iterations. The execution time of each iteration is roughly the time of a forward pass over the calibration dataset to validate the precision combination.

To demonstrate our algorithm's search efficiency, we compare it with the conventional brute-force approaches [12]–[14] on the OPT-125M model. As shown in Fig. 9, the search space for OPT-125M contains over 10,000 possible combinations, and our algorithm identifies the precision combination $[7, 7, 6, 5]$ in just 10 iterations, maintaining accuracy within 1% loss. In practice, we limit the search to 32 iterations, ensuring that time overhead remains minimal while achieving a near-optimal precision combination. By avoiding time-consuming backward propagation or complex solving processes, our algorithm operates approximately twice as fast as Omniquant [66] and ten times faster than GPTQ [24], the current SoTA methods for post-training weight-only LLM quantization.

IV. ANDA ARCHITECTURE

In this section, we first present the three key components of the Anda architecture: (a) A variable-length activation data layout in on-chip memory storage, (b) an Anda-enhanced bit-serial processing unit, and (c) a runtime bit-plane compressor for output activations. These components collectively enhance storage efficiency, computational performance, and energy

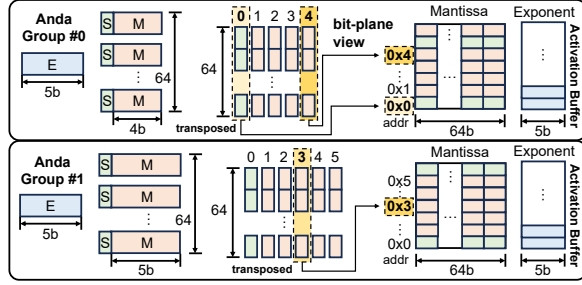


Fig. 10. The proposed bit-plane data layout scheme in memory for efficient variable-length activation data storage.

conservation. Finally, we present how these components integrate to form the overall Anda architecture, a computing system optimizing LLM inference using the Anda format.

A. Bit-plane Data Layout Scheme

Anda-based activation values feature a variable-length mantissa, necessitating careful data layout arrangement in the on-chip buffer to maintain regular memory access. Otherwise, irregular memory accesses caused by an ineffective data layout could completely undo the benefits provided by Anda.

To tackle these challenges, we propose the bit-plane data layout scheme as illustrated in Fig. 10. Unlike prior fixed-length data arrangement methods [30], [41], [61], [67], which treat each FP data element as an atomic unit, our approach separates and reorganizes the sign bit, mantissa, and exponent of FP numbers within grouped data blocks from a bit-plane view. A transposed data arrangement [48] is introduced where bits of the same significance across multiple numbers are packed together to keep the regularity of memory access. Taking the common memory bank word width into account, 64 Anda-type values are grouped to implement the bit-plane data layout scheme. As shown in Fig. 10, Group #0 shows the layout for 4-bit mantissa Anda numbers, while Group #1 presents the arrangement for 5-bit mantissa Anda numbers. The variable mantissa length only reflects on the different memory address depths, without impacting memory bandwidth utilization, and can be easily managed during address generation. Hence, in both cases, the bit-plane data layout efficiently accommodates these formats with varying lengths, maintaining consistent access patterns. Furthermore, the bit-plane organization inherently facilitates parallel processing, inspiring the design of a novel processing unit for the Anda data format to enhance LLM inference in both computing and energy efficiency.

B. Anda-enhanced Bit-serial Processing Unit

The Anda-enhanced bit-serial processing unit (APU), as depicted in Fig. 11, serves as the key computational element of the Anda architecture, embracing Anda processing element (PE) and an FP accumulator. Anda PE efficiently executes dot-product operations between variable-length Anda format activations and INT weights, seamlessly integrating with the bit-plane data layout scheme to enhance performance. The FP

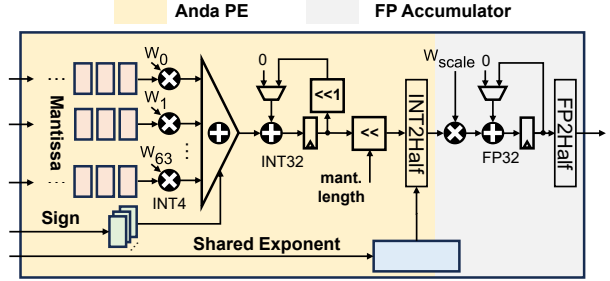


Fig. 11. The architecture of Anda-enhanced bit-serial processing unit, which enables efficient dot-product operations for Anda activations and INT weights.

accumulator follows the PE to complete the APU functionality by accumulating the cross-group dot-product results.

The computation process begins with the Anda PE storing the sign and exponent in internal registers. Concurrently, the INT weights are stored in the PE using a double-buffer design, allowing overlapped weight loading and computation to minimize loading latency. The PE then loads the bit-plane mantissas and performs computations with the INT weights. By employing bit-serial processing of mantissas, the Anda PE can adapt to Anda format data of varying lengths without additional hardware overhead.

To further optimize hardware efficiency, the Anda PE implements a first-element-then-bit-plane reduction pattern. In this approach, a partial sum is obtained for each bit-plane by accumulating all elements within that bit-plane using an adder tree. This method reduces storage requirements by storing only one partial sum per bit-plane instead of all intermediate results. It also minimizes data movement and processing overhead by performing subsequent shift operations only on the single partial sum rather than individual elements. Furthermore, it significantly reduces hardware resource consumption by using a single shared accumulator for all bit-plane accumulations.

The bit-plane partial sums are then sequentially accumulated to complete the dot-product operation. Upon completion, the Anda PE dynamically shifts the dot-product result based on the Anda mantissa length and converts it to FP16 using the shared exponent. The result is then multiplied with the group-wise scale factor of the INT weights, followed by cross-group accumulation using the FP accumulator. Finally, the accumulated FP32 result is converted to FP16 for output.

C. On-the-fly Bit-plane Compressor

The bit-plane compressor (BPC) is a critical component of the Anda architecture, enabling on-the-fly conversion of FP16 activation values into the compressed Anda format. It efficiently addresses the challenges of variable-length Anda activation storage and transfer in LLM inference by processing a large number of activation values in parallel and outputting them in a bit-serial manner.

Fig. 12 illustrates the architecture of the proposed BPC. It consists of 16 parallel lanes, each capable of processing 64 grouped FP16 values simultaneously. Within each lane, the FP field extractor decomposes the FP16 inputs into their sign,

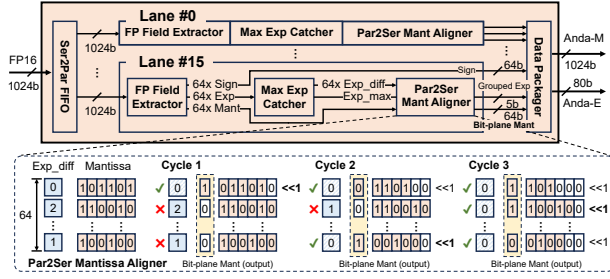


Fig. 12. The architecture of the on-the-fly bit plane compressor and the mantissa alignment process performed in the parallel-to-serial mantissa aligner.

exponent, and mantissa components. The maximum exponent catcher identifies the maximum exponent within a grouped lane, and then calculates the difference of each exponent to the shared maximum exponent.

The core of the compression process lies in the mantissa alignment performed by the parallel-to-serial mantissa aligner. As shown in Fig. 12, in each cycle, each element’s exponent difference decreases by one until it reaches zero. When the exponent difference is zero, the most significant mantissa bit of that element should be shifted out each cycle; otherwise, it remains unchanged and output zero. The shifted-out bits among each element in the lane are packed into the bit-plane aligned mantissa. This process continues for multiple cycles until the number of output bit-planes reaches the configurable mantissa length. The parallel-to-serial mantissa alignment process generates compressed bit-planes directly. The resulting bit-serial output, along with the sign bits and shared maximum exponents from all lanes, is passed to the data packager unit. This unit assembles the final compressed output in a format compatible with the proposed bit-plane compression scheme. The proposed bit-serial mantissa aligner is more area-efficient compared to existing bit-parallel aligners [32], [85], requiring only a comparator and shifter. In contrast, bit-parallel designs need multiple shifters and comparators for single-cycle dynamic shifting [15]. While our bit-serial aligner introduces some latency, it can largely overlap with APU computations, with little impact on overall system performance.

D. Overall Architecture

Fig. 13 illustrates the overall architecture of Anda, which includes the top controller, address generator, activation buffer, weight buffer, matrix computation unit (MXU), vector unit, and bit-plane compressor. The LLM inference is orchestrated as follows: ❶ Initially, the instruction memory is programmed through the I/O interface of the top controller, which governs the address generator during operation. ❷ The address generator produces read and write addresses for both activation and weight buffers. Both the activation buffer and weight buffer follow the proposed bit-plane-based data layout for efficient data handling. ❸ The MXU, featuring a 16×16 APU array, performs FP-INT GEMM operations following typical output stationary dataflow [45]. The weight data dispatcher, equipped with registers, allows overlapping weight loading and computation, broadcasting weights row-wise to each APU

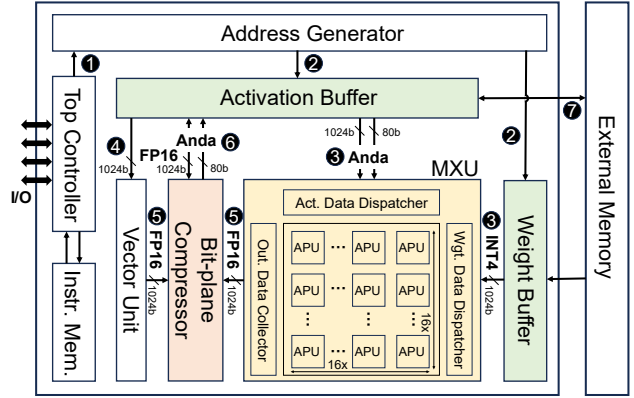


Fig. 13. Anda system architecture.

for data reuse. The activation data dispatcher supplies a bit-plane vector of activations each cycle, sequentially feeding it into the MXU and sharing it across columns to maximize input reuse and enable multiple calculations with the same input. Upon completing the GeMM computation, the output results are delivered to the BPC via the output data dispatcher.

- ④ Complementing MXU, the vector unit processes the non-linear functions of the transformer block.
- ⑤ FP16 outputs of MXU or vector unit can be optionally compressed to Anda format by the BPC, optimizing storage efficiency.
- ⑥ Processed outputs are written back to the activation buffer.
- ⑦ Finally, activation results are transferred to external memory for subsequent operations.

V. EVALUATION

A. Experimental Setup

LLM Benchmarks: To demonstrate the wide applicability of our proposed method, we benchmark various open-source LLMs using PyTorch and Hugging Face libraries. We evaluate their performance on WikiText-2 [60], Penn Treebank (PTB) [59], and C4 [65] datasets. The models used in our benchmarks range in size from 1.3B to 30B parameters and are selected from OPT [86], LLaMA [72], and LLaMA2 [73] families, enabling the effectiveness assessment of our method across different model scales and architectures.

Quantization Baselines: To validate the model accuracy when replacing FP activations with the proposed Anda format, we compare against the following SotA competitors: (a) Full-precision baseline where both activations and weights are represented in FP16. (b) Weight-only PTQ baseline using Omniquant [66] with W4A16g128 scheme, which uses 4-bit weight quantization with a group size of 128. (c) Lossless BFP baseline that adopts FIGNA’s [32] approach of using extended mantissa lengths to maintain model accuracy. (d) Aggressive BFP baseline that employs 4-bit mantissa to activations using VS-Quant [12] quantization scheme. For fair comparison in the PTQ scenario, we directly apply VS-Quant’s 4-bit data format without the typically required costly retraining. For baseline (c), (d), and our Anda method, we use baseline (b)

TABLE II
COMPARISON OF COMPUTATION METHODS OF WEIGHT-ONLY QUANTIZED LLMs
KEY METRICS: PERPLEXITY (BLACK, LOWER IS BETTER), ACCURACY DROP (RED) AND BOPs SAVING (GREEN)

	OPT-1.3B	OPT-2.7B	OPT-6.7B	LLaMA-7B	LLaMA2-7B	OPT-13B	LLaMA-13B	LLaMA2-13B	OPT-30B	
WikiText2	FP16	14.62	12.47	10.86	5.68	5.47	10.13	5.09	4.88	9.56
	OmniQuant [66]	14.88 ^{0.00%} _{1.00x}	12.65 ^{0.00%} _{1.00x}	10.96 ^{0.00%} _{1.00x}	5.77 ^{0.00%} _{1.00x}	5.59 ^{0.00%} _{1.00x}	10.20 ^{0.00%} _{1.00x}	5.17 ^{0.00%} _{1.00x}	4.95 ^{0.00%} _{1.00x}	9.62 ^{0.00%} _{1.00x}
	FIGNA [32]	14.90 ^{-0.13%} _{1.23x}	12.65 ^{0.00%} _{1.23x}	10.96 ^{0.00%} _{1.23x}	5.78 ^{-0.17%} _{1.23x}	5.60 ^{-0.18%} _{1.23x}	10.22 ^{-0.20%} _{1.23x}	5.18 ^{-0.19%} _{1.23x}	4.96 ^{-0.20%} _{1.23x}	9.61 ^{-0.10%} _{1.23x}
	VS-Quant [39]	19.04 ^{-27.96%} _{4.00x}	16.41 ^{-22.91%} _{4.00x}	12.24 ^{-11.68%} _{4.00x}	7.45 ^{-29.12%} _{4.00x}	8.26 ^{-47.76%} _{4.00x}	11.63 ^{-13.98%} _{4.00x}	6.36 ^{-23.02%} _{4.00x}	6.43 ^{-29.90%} _{4.00x}	10.67 ^{-10.91%} _{4.00x}
	Ours (0.1%)	14.91 ^{-0.20%} _{2.74x}	12.66 ^{-0.09%} _{2.91x}	10.95 ^{0.10%} _{3.10x}	5.78 ^{-0.18%} _{2.99x}	5.60 ^{-0.18%} _{1.90x}	10.21 ^{-0.10%} _{2.86x}	5.18 ^{-0.19%} _{1.88x}	4.96 ^{-0.20%} _{1.80x}	9.60 ^{-0.20%} _{3.10x}
	Ours (1%)	14.99 ^{-0.74%} _{2.95x}	12.76 ^{-0.86%} _{3.25x}	10.99 ^{-0.27%} _{3.31x}	5.82 ^{-0.87%} _{2.56x}	5.65 ^{-1.07%} _{2.56x}	10.30 ^{-0.98%} _{3.20x}	5.23 ^{-1.16%} _{2.59x}	5.00 ^{-1.01%} _{2.44x}	9.71 ^{-0.94%} _{3.31x}
PTB	FP16	16.96	15.12	13.09	8.80	20.82	12.34	8.07	28.93	11.84
	OmniQuant [66]	17.40 ^{0.00%} _{1.00x}	15.28 ^{0.00%} _{1.00x}	13.25 ^{0.00%} _{1.00x}	8.97 ^{0.00%} _{1.00x}	21.52 ^{0.00%} _{1.00x}	12.46 ^{0.00%} _{1.00x}	8.14 ^{0.00%} _{1.00x}	30.19 ^{0.00%} _{1.00x}	11.94 ^{0.00%} _{1.00x}
	FIGNA [32]	17.41 ^{-0.06%} _{1.23x}	15.28 ^{0.00%} _{1.23x}	13.26 ^{-0.08%} _{1.23x}	8.98 ^{-0.11%} _{1.23x}	21.52 ^{0.00%} _{1.23x}	12.47 ^{-0.08%} _{1.23x}	8.15 ^{-0.12%} _{1.23x}	30.19 ^{0.00%} _{1.23x}	11.95 ^{-0.08%} _{1.23x}
	VS-Quant [39]	25.64 ^{-47.36%} _{4.00x}	21.09 ^{-27.55%} _{4.00x}	15.50 ^{-16.98%} _{4.00x}	15.78 ^{-75.92%} _{4.00x}	53.46 ^{-148.42%} _{4.00x}	14.47 ^{-16.13%} _{4.00x}	12.54 ^{-54.05%} _{4.00x}	49.91 ^{-65.32%} _{4.00x}	13.47 ^{-12.81%} _{4.00x}
	Ours (0.1%)	17.41 ^{-0.06%} _{1.64x}	15.29 ^{-0.07%} _{2.13x}	13.26 ^{-0.08%} _{2.23x}	8.98 ^{-0.11%} _{2.21x}	21.48 ^{-0.20%} _{2.19x}	12.47 ^{-0.08%} _{1.84x}	8.15 ^{-0.12%} _{2.35x}	30.18 ^{-0.03%} _{2.84x}	11.96 ^{-0.17%} _{2.23x}
	Ours (1%)	17.57 ^{-0.98%} _{2.31x}	15.42 ^{-0.91%} _{2.70x}	13.35 ^{-0.75%} _{2.86x}	9.05 ^{-0.89%} _{2.54x}	21.61 ^{-0.42%} _{2.39x}	12.58 ^{-0.96%} _{2.67x}	8.22 ^{-0.98%} _{2.67x}	30.41 ^{-0.73%} _{2.92x}	12.03 ^{-0.75%} _{2.86x}
C4	FP16	14.72	13.16	11.74	7.08	6.97	11.20	6.61	6.47	10.69
	OmniQuant [66]	15.03 ^{0.00%} _{1.00x}	13.38 ^{0.00%} _{1.00x}	11.85 ^{0.00%} _{1.00x}	7.21 ^{0.00%} _{1.00x}	7.12 ^{0.00%} _{1.00x}	11.29 ^{0.00%} _{1.00x}	6.69 ^{0.00%} _{1.00x}	6.56 ^{0.00%} _{1.00x}	10.75 ^{0.00%} _{1.00x}
	FIGNA [32]	15.04 ^{-0.07%} _{1.23x}	13.38 ^{0.00%} _{1.23x}	11.86 ^{-0.08%} _{1.23x}	7.22 ^{-0.14%} _{1.23x}	7.12 ^{0.00%} _{1.23x}	11.29 ^{0.00%} _{1.23x}	6.70 ^{-0.15%} _{1.23x}	6.57 ^{-0.15%} _{1.23x}	10.75 ^{0.00%} _{1.23x}
	VS-Quant [39]	19.00 ^{-26.41%} _{4.00x}	16.65 ^{-19.64%} _{4.00x}	13.13 ^{-10.80%} _{4.00x}	8.89 ^{-23.30%} _{4.00x}	10.36 ^{-45.51%} _{4.00x}	12.64 ^{-11.96%} _{4.00x}	7.85 ^{-17.34%} _{4.00x}	8.35 ^{-27.29%} _{4.00x}	11.91 ^{-10.79%} _{4.00x}
	Ours (0.1%)	15.05 ^{-0.13%} _{1.86x}	13.40 ^{-0.15%} _{2.11x}	11.87 ^{-0.17%} _{2.23x}	7.22 ^{-0.14%} _{2.16x}	7.13 ^{-0.14%} _{2.04x}	11.30 ^{-0.09%} _{2.23x}	6.70 ^{-0.15%} _{2.16x}	6.57 ^{-0.14%} _{2.14x}	10.76 ^{-0.09%} _{2.31x}
	Ours (1%)	15.17 ^{-0.93%} _{2.43x}	13.51 ^{-0.96%} _{2.86x}	11.97 ^{-1.01%} _{3.05x}	7.28 ^{-0.97%} _{2.70x}	7.19 ^{-0.98%} _{2.67x}	11.37 ^{-0.71%} _{2.86x}	6.74 ^{-0.75%} _{2.70x}	6.62 ^{-0.91%} _{2.70x}	10.85 ^{-0.93%} _{3.09x}

* For fair comparison in the PTQ scenario, we directly apply VS-Quant's 4-bit data format without the costly retraining typically required by this method.

as the starting point, and the group size of BFP activations is uniformly set to 64.

Hardware Baselines: To further highlight the advantages of the Anda scheme, we compare the Anda hardware architecture against several SotA platforms: (a) FP-FP: A spatial accelerator based on FP16 Tensor Cores [84], representative of current GPU architectures [69]. (b) FP-INT: An enhanced Tensor Core-based accelerator featuring specialized FP-INT computation units. (c) iFPU [42]: A spatial accelerator employs bit-serial computation for INT weights and dynamically converts FP activations to BFP format with extended mantissa before computation. (d) FIGNA [32]: An evolution of (c) leveraging bit-parallel computation and optimized mantissa length in BFP to enhance computational efficiency. All systems are configured to have the same operating clock frequency of 285 MHz, equivalent peak throughput, and on-chip memory resources to ensure a fair comparison [68].

Evaluation Methodologies: Our comprehensive assessment encompasses both model accuracy and hardware efficiency, where we focus on optimizing the dominated FP-INT GeMM operations, keeping the others, e.g., KV cache [56], in FP16. For model accuracy, we employ three key metrics: (a) perplexity (PPL) [33] using a 2048 sequence length, where lower values indicate higher model accuracy; (b) relative accuracy loss of FIGNA, VS-Quant, and our Anda method compared to OmniQuant, which demonstrates the accuracy drop during BFP conversion in the deployment process; and (c) bit-level operations (BOPs) [1] reduction, quantifying the decrease in theoretical model computation. Here we consider one FP16-INT4 operation equivalent to 64 BOPs, as this approximates the bit-level computational complexity of an FP16-INT4 multiply-accumulate operation. Hardware performance evaluation spans two levels: at the PE level, we analyze area and power along with corresponding efficiencies; at the system

level, we compare Anda with baseline architectures, focusing on speedup and energy efficiency, when performing FP-INT GeMMs where the FP activations can be drop-in replaced by Anda data format. In alignment with prior studies [27], [32], [42], [79], system-level evaluation uses LLMs with a batch size of 1 and the maximum acceptable input sequence length under the WikiText2 dataset. PE-level baselines and Anda system are implemented in SystemVerilog RTL and synthesized using Cadence Genus [6] at 16nm technology node, operating at a clock frequency of 285 MHz and 0.8 V nominal voltage. Power evaluation is performed using value change dump (VCD) files generated from synthesized netlist simulations and analyzed by Genus. A cycle-accurate simulator, rigorously verified against functional simulations, assesses the energy and performance of Anda and baseline accelerators. HBM2 memory is modeled with an access energy of 3.9 pJ/bit and a bandwidth of 256 GB/s [36].

B. Inference Accuracy

We evaluate inference accuracy on validation datasets using the Anda format explored by adaptive precision search algorithm across all benchmarks. For each benchmark, 128 random sequences of length 2048 are sampled from the training dataset for calibration [66]. We also limit the adaptive precision search algorithm to 32 iterations. Note that Anda is capable of adapting mantissa bits according to the user-defined accuracy tolerance. Therefore, we report results for two accuracy constraints: 0.1% representing minimal loss and 1% representing acceptable loss for most scenarios.

Table II compares Anda's performance against baseline quantization methods, where PPL values are shown in black, the relative accuracy drop is displayed in red, and the BOPs reduction is presented in green. Note that the occasional slight exceedance of the validation accuracy loss over the

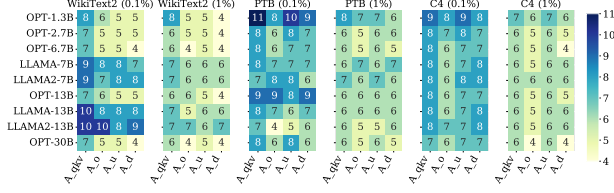


Fig. 14. Identified best precision combinations of various LLMs on different datasets given different accuracy tolerances.

constraint is normal for Anda due to differences between the calibration and validation datasets. The results demonstrate that Anda achieves significant BOPs reductions while maintaining accuracy close to the target constraints. For instance, on the WikiText2 dataset, Anda achieves $1.80\sim 3.10\times$ and $2.44\sim 3.31\times$ BOPs reduction under 0.1% and 1% accuracy loss, respectively, flexibly meeting varying accuracy-efficiency requirements. Compared to FIGNA, which yields a $1.23\times$ BOPs reduction with shorter bit-width multiplication, Anda further achieves $1.46\sim 2.69\times$ BOPs reductions at similar accuracy loss levels by leveraging different mantissa lengths for activation tensors. In contrast to VS-Quant, a BFP method requiring retraining, directly deploying it leads to severe accuracy degradation despite achieving $4\times$ BOPs reduction. For example, VS-Quant suffers a 27.96% accuracy loss on OPT-1.3B with WikiText2. Anda, however, achieves a much better accuracy-efficiency balance, obtaining nearly $3\times$ BOPs reduction with only a 0.74% accuracy loss in the same scenario. Anda's consistent performance improvements across various models and datasets showcase the effectiveness and robust generalization of our adaptive precision search algorithm.

Fig. 14 presents the best precision combination driven by 0.1% and 1% relative accuracy loss, revealing the patterns of quantization precision choices for different activation parts across models. The precision combinations vary under different accuracy constraints, showing the importance of the adaptive precision combination search algorithm in automatically adjusting the mantissa lengths based on LLM module sensitivity. Examining the module types reveals that A_{qkv} , involved with the Q , K , and V projection layers, prefers higher precision due to higher sensitivity, while A_u and A_d in the feed-forward layers, especially A_d , can be more aggressively quantized to lower precision, demonstrating higher tolerance.

C. PE-level Evaluation

We quantitatively compare the proposed Anda PE with common FP-FP units [58], enhanced FP-INT units, and dedicated PE units from iFPU [42] and FIGNA [32], respectively. We also introduce FIGNA-M11 and FIGNA-M8 as baselines, representing bit-parallel PEs with 11-bit and 8-bit mantissas that achieve 0.1% and 1% accuracy degradation targets, respectively, based on the results from Fig. 14. Here, $M(x)$ denotes the number of preserved mantissa bits. To ensure an equitable evaluation, all PEs are configured with equal computational throughput per cycle. We process an identical

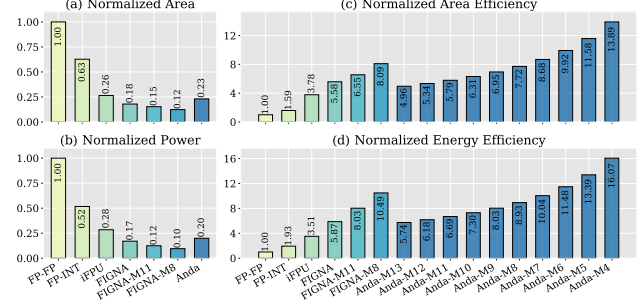


Fig. 15. PE-level comparison in terms of area, power, area efficiency, and energy efficiency. All data are normalized to the GPU-like FP-FP baseline.

dot product workload across different PEs to measure area efficiency (TOPS/mm²) and energy efficiency (TOPS/W).

Fig. 15 (a) and (b) show the area and power consumption of Anda and baseline PEs. Anda presents significant reductions, consuming less than 60% of the power and area compared to FP-FP and FP-INT PEs. This is primarily due to shared exponents, which eliminate complex alignment and normalization processes. Compared to iFPU [42], Anda offers 12% and 29% reductions in area and power, respectively, by avoiding high-overhead ultra-wide multipliers and registers needed for maintaining FP16 precision. While Anda incurs a 27% power and 18% area overhead compared to FIGNA due to its bit-serial structure, its adaptive precision capability can significantly reduce execution time, leading to higher efficiency. Fig. 15 (c) and (d) further exhibit superior area and energy efficiency of Anda PE with variable-length mantissas. Referring back to Fig. 14, the retained mantissa lengths of Anda typically range between 4~8 bits with negligible 1% accuracy impact, resulting in the area and energy efficiency improvements of $1.38\sim 2.48\times$ and $1.52\sim 2.74\times$ over FIGNA, respectively. Moreover, comparing FIGNA and Anda at fixed mantissa lengths, Anda introduces some control logic overhead due to its bit-serial design. At 11 bits, Anda has 12% and 17% lower area and energy efficiency against FIGNA-M11; at 8 bits, it's 5% and 15% lower against FIGNA-M8. However, Anda's ability to dynamically adjust mantissa lengths based on model accuracy requirements allows it to potentially achieve higher utilization at the system level, which will be analyzed in the next subsection.

D. System-level Evaluation

Fig. 16 compares system-level speedup, area efficiency, and energy efficiency between Anda and several baselines across various LLM models. We also introduce bit-parallel FIGNA-M11 and FIGNA-M8 as baselines for 0.1% and 1% accuracy loss. Anda enables precision-scalable inference within a single hardware architecture, in contrast to FIGNA's separate implementations for each precision level.

Speedup: Anda, utilizing the precision combinations identified in Fig. 14, implements scalable computation and achieves $2.14\times$ and $2.49\times$ speedups on average over the GPU-like FP-FP baseline at 0.1% and 1% accuracy loss, respectively. Com-

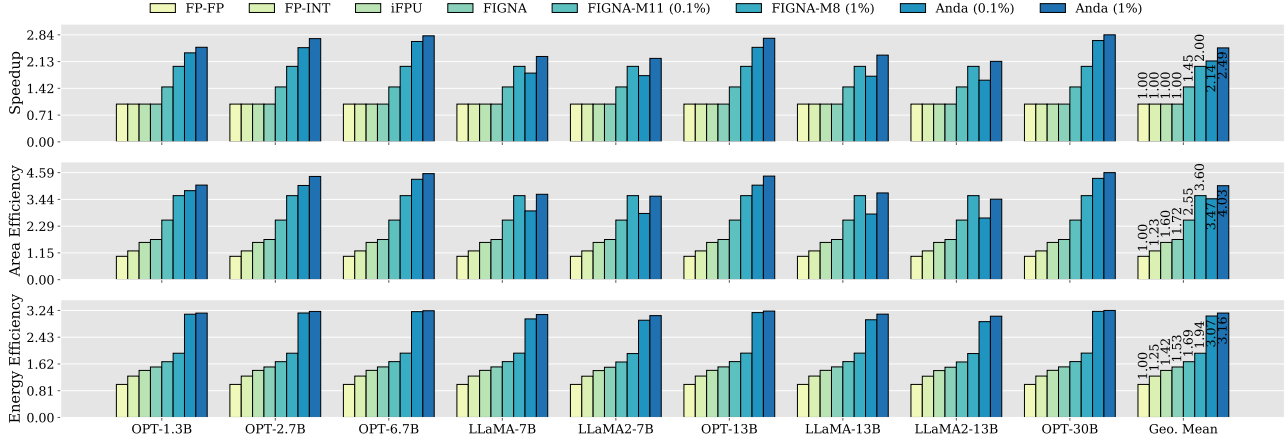


Fig. 16. Speedup, area efficiency, and energy efficiency comparison across accelerators on WikiText2. All data are aligned to the GPU-like FP-FP baseline.

pared to the corresponding FIGNA variants, Anda achieves $1.48\times$ and $1.25\times$ higher acceleration, benefiting from efficient utilization of varied mantissa precisions across tensor types.

Area Efficiency: Anda improves area efficiency by $3.47\times$ and $4.03\times$ over the GPU-like FP-FP baseline at 0.1% and 1% loss, respectively, due to two factors: (a) shared exponent design simplifies alignment operations, improving computational unit efficiency; (b) bit-serial design fully utilizes mantissa widths of different tensor types. Notably, in 1% loss with LLaMA models, FIGNA-M8’s area efficiency rivals or slightly exceeds Anda due to its alignment with 8-bit precision, where bit-parallel designs excel. However, Anda’s scalable computation outperforms FIGNA by adopting more aggressive bit-widths in OPT models.

Energy Efficiency: Anda achieves a $3.07\times$ improvement over the GPU-like FP-FP baseline at 0.1% accuracy loss, increasing to $3.16\times$ at 1% loss tolerance. Unlike iFPU [42] and FIGNA [32], which solely optimize energy during computation, Anda’s bit-serial architecture skips redundant mantissa bit calculations to improve computational utilization, and the BPC compresses output, reducing memory access. FIGNA-M11 and FIGNA-M8 use reduced mantissa bit-parallel designs to improve computational efficiency, but rely on FP16 storage, leading to frequent data conversions, which offsets energy gains. Fig. 17 further presents that compared to the GPU-like FP-FP baseline on the LLaMA-13B model, Anda reduces energy consumption by 90%, 54%, and 50% for computation, SRAM, and DRAM access, respectively. While FIGNA achieves similar compute efficiency, Anda’s architecture avoids redundant computations and FP-to-BFP conversion, reducing energy further. Moreover, Anda’s bit-plane storage scheme and BPC compression reduce memory access overhead, improving SRAM and DRAM energy efficiency by $2.2\times$ and $2.0\times$ compared to FIGNA.

E. Power and Area Breakdown

We conduct a detailed hardware analysis of the Anda architecture for LLaMA-13B inference within 1% accuracy loss.

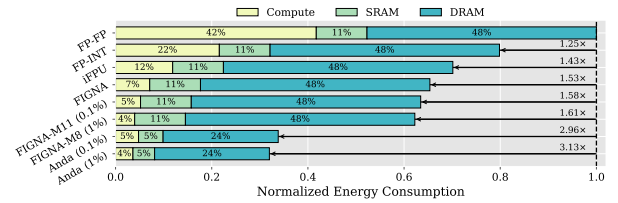


Fig. 17. Energy breakdown of Anda in contrast with the baseline accelerators. Energy consumption during the LLaMA-13B inference is evaluated.

TABLE III
AREA AND POWER CHARACTERISTICS OF ANDA

Component	Setup	Area [mm ²]	Power [mW]
MXU	16×16 APUs	0.41 (18.89%)	54.34 (66.94%)
BPC	16 Lanes	0.07 (3.23%)	1.06 (1.31%)
Vector Unit	64 FPUs	0.05 (2.30%)	0.87 (1.07%)
Activation Buffer	1MB (Mant.) + 0.125MB (Exp.)	0.87 (40.09%)	16.94 (20.87%)
Weight Buffer	1MB	0.80 (36.87%)	7.96 (9.81%)
Others	Top controller	0.01 (0.46%)	0.01 (0.00%)
Total		2.17 (100.00%)	81.18 (100.00%)

Table III presents the area breakdown and power distribution. Operating at 285 MHz and 0.8 V, Anda occupies 2.17 mm² with 81.18 mW power consumption. The MXU, serving as the core computing component of the Anda architecture, consumes 66.94% of the total power despite occupying only 18.89% of the area. The BPC unit, which enables efficient online compression from the full-precision FP outputs to the Anda format, costs a small portion of the total area (3.23%) and power consumption (1.31%). On-chip SRAM is the primary area contributor, with the activation buffer and weight buffer accounting for 40.09% and 36.87% of the total area, respectively. Their power consumption ratios are relatively low at 20.87% and 9.81% because of efficient data reuse within the Anda system.

F. Accuracy-Performance Trade-off

This section explores speedup and energy efficiency improvements of the Anda system over the FP-FP baseline with

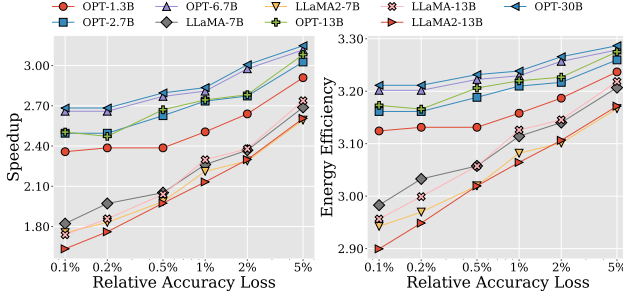


Fig. 18. Speedup and energy efficiency improvement of Anda over FP-FP baseline towards various acceptable accuracy losses.

accuracy loss constraints ranging from 0.1% to 5%. As shown in Fig. 18, using LLaMA-13B as an example, Anda achieves a $1.73\times$ speedup and $2.95\times$ energy efficiency improvement with only 0.1% accuracy loss, increasing to $2.74\times$ and $3.22\times$, respectively, when the constraint is relaxed to 5%. All models exhibit significant acceleration and efficiency gains as the tolerated accuracy loss increases. Notably, OPT and LLaMA models exhibit distinct characteristics when using the Anda format. This stems from OPT’s lower sensitivity to bit-width reductions, allowing the use of shorter mantissa bit-widths with minimal accuracy sacrifice. Consequently, under tighter accuracy constraints, e.g., 0.1%~0.5%, OPT models achieve greater speedups and energy efficiency improvements compared to LLaMA models. However, as accuracy constraints relax, their performance gains gradually converge. By integrating the adaptive precision combination search algorithm with the Anda format, our architecture achieves flexible balancing of system performance and accuracy across diverse practical application scenarios, enabling efficient LLM inference under different LLM architectures and varying requirements.

VI. RELATED WORKS AND DISCUSSIONS

Bit-serial & bit-parallel computing. Bit-serial computing [2], [7], [37], [46], [55], [68], [75] has long been explored in DNN acceleration, offering flexibility for variable precision computations. However, most prior work focuses on INT operations, limiting applicability to LLMs with FP activations. Approaches like Bitlet [55] and Bitlet-X [7] explore FP-based bit-serial computing but introduce complex hardware and dataflow designs due to bit-interleave schemes. In contrast, our Anda simplifies alignment overhead using variable-length grouped activation encoding, leading to a more efficient hardware design. Although bit-serial computing typically has lower area efficiency and higher latency than bit-parallel approaches [41], [61], [67], [85] due to complex timing control logic, it offers higher utilization across precision-scalable scenarios. While Anda uses bit-serial units, its design principles can benefit bit-parallel computing as well. For instance, the proposed bit-precision combination search method can rapidly determine the required precision for bit-parallel applications, potentially improving efficiency while maintaining accuracy.

PTQ & quantization-aware training (QAT). Quantization, a key compression technique, is generally categorized into PTQ and QAT. In the LLM era, PTQ [24], [51], [66], [79] is more popular, efficiently producing deployable models with good accuracy in hours on a single GPU. In contrast, QAT for LLMs [9], [21], [53], [74], while potentially more accurate, is often impractical due to its extensive computational demands, often requiring multi-GPU systems and hundreds of training hours. Anda adopts the PTQ approach, swiftly allocating mantissa lengths for FP activations, and can be integrated into existing deployment pipelines with minimal overhead. Future research could explore using Anda for QAT, potentially enhancing accuracy while reducing computational costs.

KV cache optimization. In long-context scenarios, KV cache [82], which stores attention keys and values during generation to avoid re-computation, becomes a memory and speed bottleneck as its size grows linearly with the number of tokens. Various techniques have been proposed to tackle this challenge including quantization [29], [54], eviction [10], [88], sliding window [20], [80], and merging strategies [40], [87]. Anda, while focusing on FP activation compression, could synergize with these KV cache optimizations to significantly accelerate long-context LLM inference.

VII. CONCLUSION

This work presents Anda, a variable-length grouped activation data format that addresses energy and performance bottlenecks of weight-only quantized large language model (LLM) inference by exploiting redundancy in floating point activations across different models and their modules. To fully harness the potential of Anda, we develop an iterative post-training algorithm that optimizes bit-width allocation across LLM modules, balancing accuracy, energy efficiency, and inference speed. We design complementary hardware optimizations to maximize the benefits of Anda, including a bit-plane-based data organization scheme in memory, Anda-enhanced bit-serial processing units, and a runtime bit-plane compressor. Our evaluations show that Anda achieves a $2.4\times$ speedup, a $4.0\times$ enhancement in area efficiency, and a $3.1\times$ improvement in energy efficiency on average for popular LLMs compared to the GPU-like FP-FP baseline. With its adaptability across various application scenarios and performance requirements, Anda enables efficient LLM inference in diverse deployment environments, paving the way for broader adoption of LLMs in resource-constrained settings.

ACKNOWLEDGEMENTS

This project has been partly funded by the National Key R&D Program of China under Grant 2022YFB4400600, the European Research Council (ERC) under grant agreement No. 101088865, the Flanders AI Research Program, the KU Leuven Hercules program, and the China Scholarship Council Program (Grant No. 202306190235). We would like to sincerely thank Mengzhao Chen at HKU, Zhe Wang at HONOR, Zhi Zhang at SenseTime for the helpful discussion, and the anonymous reviewers for their constructive feedback.

REFERENCES

- [1] D. Abati, H. Ben Yahia, M. Nagel, and A. Habibian, "Resq: Residual quantization for video perception," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 17 119–17 129.
- [2] J. Albericio, A. Delmás, P. Judd, S. Sharify, G. O'Leary, R. Genov, and A. Moshovos, "Bit-pragmatic deep neural network computing," in *Proceedings of the 50th annual IEEE/ACM international symposium on microarchitecture (MICRO)*, 2017, pp. 382–394.
- [3] Y. Bahri, E. Dyer, J. Kaplan, J. Lee, and U. Sharma, "Explaining neural scaling laws," *Proceedings of the National Academy of Sciences (PNAS)*, vol. 121, no. 27, p. e2311878121, 2024.
- [4] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "Longbench: A bilingual, multitask benchmark for long context understanding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (ACL)*, 2024, pp. 3119–3137.
- [5] P. Bhattacharya, V. K. Prasad, A. Verma, D. Gupta, A. Sapsomboon, W. Viriyasitavat, and G. Dhiman, "Demystifying chatgpt: An in-depth survey of openai's robust large language models," *Archives of Computational Methods in Engineering*, pp. 1–44, 2024.
- [6] Cadence, "Genus synthesis solution," https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/synthesis/genus-synthesis-solution.html, 2024, online; accessed 2024-07-16.
- [7] L. Chang, H. Lu, C. Li, X. Zhao, Z. Hu, J. Zhou, and X. Li, "General purpose deep learning accelerator based on bit interleaving," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 2023.
- [8] J. Chee, Y. Cai, V. Kuleshov, and C. M. De Sa, "Quip: 2-bit quantization of large language models with guarantees," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2024.
- [9] M. Chen, W. Shao, P. Xu, J. Wang, P. Gao, K. Zhang, Y. Qiao, and P. Luo, "Efficientqat: Efficient quantization-aware training for large language models," *arXiv preprint arXiv:2407.11062*, 2024.
- [10] Y. Chen, G. Wang, J. Shang, S. Cui, Z. Zhang, T. Liu, S. Wang, Y. Sun, D. Yu, and H. Wu, "Nacl: A general and effective kv cache eviction framework for llm at inference time," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (ACL)*, 2024, pp. 7913–7926.
- [11] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov, R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. S. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel, "Palm: Scaling language modeling with pathways," *Journal of Machine Learning Research (JMLR)*, vol. 24, no. 240, pp. 1–113, 2023.
- [12] S. Dai, R. Venkatesan, M. Ren, B. Zimmer, W. Dally, and B. Khailany, "Vs-quant: Per-vector scaled quantization for accurate low-precision neural network inference," *Proceedings of Machine Learning and Systems (MLSys)*, vol. 3, pp. 873–884, 2021.
- [13] B. Darvish Rouhani, D. Lo, R. Zhao, M. Liu, J. Fowers, K. Ovtcharov, A. Vinogradsky, S. Massengill, L. Yang, R. Bittner, A. Forin, H. Zhu, T. Na, P. Patel, S. Che, L. Chand Koppaka, X. Song, S. Som, K. Das, S. T. S. Reinhardt, S. Lanka, E. Chung, and D. Burger, "Pushing the limits of narrow precision inferencing at cloud scale with microsoft floating point," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 10271–10281.
- [14] B. Darvish Rouhani, R. Zhao, V. Elango, R. Shafipour, M. Hall, M. Mesmakhosroshahi, A. More, L. Melnick, M. Golub, G. Varatkar, L. Shao, G. Kolhe, D. Melts, J. Klar, R. L'Heureux, M. Perry, D. Burger, E. Chung, Z. S. Deng, S. Naghshineh, J. Park, and M. Naumov, "With shared microexponents, a little shifting goes a long way," in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023, pp. 1–13.
- [15] S. Das and S. P. Khatrri, "A timing-driven approach to synthesize fast barrel shifters," *IEEE Transactions on Circuits and Systems II: Express Briefs (TCAS-II)*, vol. 55, no. 1, pp. 31–35, 2008.
- [16] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Llm.int8(): 8-bit matrix multiplication for transformers at scale," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 30 318–30 332, 2022.
- [17] T. Dettmers and L. Zettlemoyer, "The case for 4-bit precision: k-bit inference scaling laws," in *International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 7750–7774.
- [18] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "Hawq: Hessian aware quantization of neural networks with mixed-precision," in *Proceedings of the IEEE/CVF international conference on computer vision (ICCV)*, 2019, pp. 293–302.
- [19] M. Drumond, T. Lin, M. Jaggi, and B. Falsafi, "Training dnns with hybrid block floating point," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.
- [20] H. Duanmu, Z. Yuan, X. Li, J. Duan, X. Zhang, and D. Lin, "Skvq: Sliding-window key and value cache quantization for large language models," in *First Conference on Language Modeling (COLM)*, 2024.
- [21] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko, and D. Alistarh, "Extreme compression of large language models via additive quantization," in *International Conference on Machine Learning (ICML)*. PMLR, 2024.
- [22] H. Fan, H.-C. Ng, S. Liu, Z. Que, X. Niu, and W. Luk, "Reconfigurable acceleration of 3d-cnns for human action recognition with block floating-point representation," in *28th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2018, pp. 287–2877.
- [23] H. Fan, G. Wang, M. Ferianc, X. Niu, and W. Luk, "Static block floating-point quantization for convolutional neural networks on fpga," in *International Conference on Field-Programmable Technology (ICFPT)*. IEEE, 2019, pp. 28–35.
- [24] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Optq: Accurate quantization for generative pre-trained transformers," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- [25] R. Gong, Y. Yong, S. Gu, Y. Huang, Y. Zhang, X. Liu, and D. Tao, "Llmc: Benchmarking large language model quantization with a versatile compression toolkit," *arXiv preprint arXiv:2405.06001*, 2024.
- [26] C. Guo, B. Lou, X. Liu, D. Boland, P. H. Leong, and C. Zhuo, "Boost: block minifloat-based on-device cnn training accelerator with transfer learning," in *IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.
- [27] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023, pp. 1–15.
- [28] S. Han, J. Kang, H. Mao, Y. Hu, X. Li, Y. Li, D. Xie, H. Luo, S. Yao, Y. Wang, H. Yang, and W. B. J. Dally, "Ese: Efficient speech recognition engine with sparse lstm on fpga," in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2017, pp. 75–84.
- [29] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "Kvquant: Towards 10 million context length llm inference with kv cache quantization," *arXiv preprint arXiv:2401.18079*, 2024.
- [30] L. Huang, C. Fang, Q. Li, J. Lin, and Z. Wang, "A precision-scalable risc-v dnn processor with on-device learning capability at the extreme edge," in *29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 927–932.
- [31] Q. Huang, P.-A. Tsai, J. S. Emer, and A. Parashar, "Mind the gap: Attainable data movement and operational intensity bounds for tensor algorithms," in *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.
- [32] J. Jang, Y. Kim, J. Lee, and J.-J. Kim, "Figma: Integer unit-based accelerator design for fp-int gemm preserving numerical accuracy," in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 760–773.
- [33] F. Jelinek, R. L. Mercer, L. R. Bahl, and J. K. Baker, "Perplexity—a measure of the difficulty of speech recognition tasks," *The Journal of the Acoustical Society of America (JASA)*, vol. 62, no. S1, pp. S63–S63, 1977.
- [34] Y. Jeon, C. Lee, E. Cho, and Y. Ro, "Mr. biq: Post-training non-uniform quantization based on minimizing the reconstruction error," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 12 329–12 338.

- [35] Y. Jeon, B. Park, S. J. Kwon, B. Kim, J. Yun, and D. Lee, "Biggemm: matrix multiplication with lookup table for binary-coding-based quantized dnn," in *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, 2020, pp. 1–14.
- [36] N. P. Jouppi, D. Hyun Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. Ma, X. Ma, T. Norrie, N. Patil, S. Prasad, C. Young, Z. Zhou, and D. Patterson, "Ten lessons from three generations shaped google's tpuv4i: Industrial product," in *ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 1–14.
- [37] P. Judd, J. Albericio, T. Hetherington, T. M. Aamodt, and A. Moshovos, "Stripes: Bit-serial deep neural network computing," in *49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–12.
- [38] K. S. Kalyan, "A survey of gpt-3 family large language models including chatgpt and gpt-4," *Natural Language Processing Journal*, p. 100048, 2023.
- [39] B. Keller, R. Venkatesan, S. Dai, S. G. Tell, B. Zimmer, C. Sakr, W. J. Dally, C. T. Gray, and B. Khailany, "A 95.6-tops/w deep learning inference accelerator with per-vector scaled 4-bit quantization in 5 nm," *IEEE Journal of Solid-State Circuits (JSSC)*, vol. 58, no. 4, pp. 1129–1141, 2023.
- [40] J.-H. Kim, J. Yeom, S. Yun, and H. O. Song, "Compressed context memory for online language model interaction," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [41] Y. Kim, C. Oh, J. Hwang, W. Kim, S. Oh, Y. Lee, H. Sharma, A. Yazdanbakhsh, and J. Park, "Dacapo: Accelerating continuous learning in autonomous systems for video analytics," in *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.
- [42] Y. Kim, J. Jang, J. Lee, J. Park, J. Kim, B. Kim, B. park, S. J. Kwon, D. Lee, and J.-J. Kim, "Winning both the accuracy of floating point activation and the simplicity of integer arithmetic," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- [43] I. Koryakovskiy, A. Yakovleva, V. Buchnev, T. Isaev, and G. Odínokikh, "One-shot model for mixed-precision quantization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7939–7949.
- [44] U. Köster, T. J. Webb, X. Wang, M. Nassar, A. K. Bansal, W. H. Constable, O. H. Elibol, S. Gray, S. Hall, L. Hornof, A. Khosrowshahi, C. Kloss, R. J. Pai, and N. Rao, "Flexpoint: An adaptive numerical format for efficient training of deep neural networks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [45] J. Lee, W. Lee, and J. Sim, "Tender: Accelerating large language models via tensor decomposition and runtime requantization," in *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.
- [46] A. Li, H. Mo, W. Zhu, Q. Li, S. Yin, S. Wei, and L. Liu, "Bitcluster: Fine-grained weight quantization for load-balanced bit-serial neural network accelerators," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 41, no. 11, pp. 4747–4757, 2022.
- [47] L. Li, Q. Li, B. Zhang, and X. Chu, "Norm tweaking: High-performance low-bit quantization of large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 38, no. 17, 2024, pp. 18 536–18 544.
- [48] T. Li, W. Romaszkan, S. Pamarti, and P. Gupta, "Geo: Generation and execution optimized stochastic computing accelerator for neural networks," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 689–694.
- [49] Z. Li, A. Lu, Y. Xie, Z. Kong, M. Sun, H. Tang, Z. J. Xue, P. Dong, C. Ding, Y. Wang, X. Lin, and Z. Fang, "Quasar-vit: Hardware-oriented quantization-aware architecture search for vision transformers," in *Proceedings of the 38th ACM International Conference on Supercomputing (ICS)*, 2024, pp. 324–337.
- [50] X. Lian, Z. Liu, Z. Song, J. Dai, W. Zhou, and X. Ji, "High-performance fpga-based cnn accelerator with block-floating-point arithmetic," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems (TVLSI)*, vol. 27, no. 8, pp. 1874–1885, 2019.
- [51] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for llm compression and acceleration," in *The Seventh Annual Conference on Machine Learning and Systems (MLSys)*, 2024.
- [52] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "Qserve: W4a8kv4 quantization and system co-design for efficient llm serving," *arXiv preprint arXiv:2405.04532*, 2024.
- [53] Z. Liu, B. Oguz, C. Zhao, E. Chang, P. Stock, Y. Mehdad, Y. Shi, R. Krishnamoorthi, and V. Chandra, "Llm-qat: Data-free quantization aware training for large language models," *arXiv preprint arXiv:2305.17888*, 2023.
- [54] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "Kivi: A tuning-free asymmetric 2bit quantization for kv cache," in *Forty-first International Conference on Machine Learning (ICML)*, 2024.
- [55] H. Lu, L. Chang, C. Li, Z. Zhu, S. Lu, Y. Liu, and M. Zhang, "Distilling bit-level sparsity parallelism for general purpose deep learning acceleration," in *54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 963–976.
- [56] S. Luohe, H. Zhang, Y. Yao, Z. Li *et al.*, "Keep the cost down: A review on methods to optimize llm's kv-cache consumption," in *First Conference on Language Modeling (COLM)*, 2024.
- [57] S. Ma, C. Fang, H. Shao, and Z. Wang, "Efficient arbitrary precision acceleration for large language models on gpu tensor cores," *arXiv preprint arXiv:2409.17870*, 2024.
- [58] S. Mach, F. Schuiki, F. Zaruba, and L. Benini, "Fpnew: An open-source multifunction floating-point unit architecture for energy-proportional transprecision computing," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems (TVLSI)*, vol. 29, no. 4, pp. 774–787, 2020.
- [59] M. Marcus, G. Kim, M. A. Marcinkiewicz, R. MacIntyre, A. Bies, M. Ferguson, K. Katz, and B. Schasberger, "The penn treebank: Annotating predicate argument structure," in *Human Language Technology: Proceedings of a Workshop held at Plainsboro, New Jersey, March 8-11, 1994*, 1994.
- [60] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *International Conference on Learning Representations (ICLR)*, 2017.
- [61] S.-H. Noh, J. Koo, S. Lee, J. Park, and J. Kung, "Flexblock: A flexible dnn training accelerator with multi-mode block floating point support," *IEEE Transactions on Computers (TC)*, vol. 72, no. 9, pp. 2522–2535, 2023.
- [62] NVIDIA, "Cutlass," <https://github.com/NVIDIA/cutlass>, 2024, online; accessed 2024-07-03.
- [63] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, R. Avila, I. Babuschkin, S. Balaji, V. Balcom, P. Baltescu, H. Bao, M. Bavarian, J. Belgum, I. Bello, J. Berdine, G. Bernadett-Shapiro, C. Berner, L. Bogdonoff, O. Boiko, M. Boyd, A.-L. Brakman, G. Brockman, T. Brooks, M. Brundage, K. Button, T. Cai, R. Campbell, A. Cann, B. Carey, C. Carlson, R. Carmichael, B. Chan, C. Chang, F. Chantzis, D. Chen, S. Chen, R. Chen, J. Chen, M. Chen, B. Chess, C. Cho, C. Chu, H. W. Chung, D. Cummings, J. Currier, Y. Dai, C. Decareaux, T. Degry, N. Deutsch, D. Deville, A. Dhar, D. Dohan, S. Dowling, S. Dunning, A. Ecoffet, A. Eleti, T. Eloundou, D. Farhi, L. Fedus, N. Felix, S. P. Fishman, J. Forte, I. Fulford, L. Gao, E. Georges, C. Gibson, V. Goel, T. Gogineni, G. Goh, R. Gontijo-Lopes, J. Gordon, M. Grafstein, S. Gray, R. Greene, J. Gross, S. S. Gu, Y. Guo, C. Hallacy, J. Han, J. Harris, Y. He, M. Heaton, J. Heidecke, C. Hesse, A. Hickey, W. Hickey, P. Hoeschele, B. Houghton, K. Hsu, S. Hu, X. Hu, J. Huizinga, S. Jain, S. Jain, J. Jang, A. Jiang, R. Jiang, H. Jin, D. Jin, S. Jomoto, B. Jonn, H. Jun, T. Kaftan, Łukasz Kaiser, A. Kamali, I. Kanitscheider, N. S. Keskar, T. Khan, L. Kilpatrick, J. W. Kim, C. Kim, Y. Kim, J. H. Kirchner, J. Kiros, M. Knight, D. Kokotajlo, Łukasz Kondraciuk, A. Kondrich, A. Konstantinidis, K. Kosic, G. Krueger, V. Kuo, M. Lampe, I. Lan, T. Lee, J. Leike, J. Leung, D. Levy, C. M. Li, R. Lim, M. Lin, S. Lin, M. Litwin, T. Lopez, R. Lowe, P. Lue, A. Makanju, K. Malfacini, S. Manning, T. Markov, Y. Markovski, B. Martin, K. Mayer, A. Mayne, B. McGrew, S. M. McKinney, C. McLeavey, P. McMillan, J. McNeil, D. Medina, A. Mehta, J. Menick, L. Metz, A. Mishchenko, P. Mishkin, V. Monaco, E. Morikawa, D. Mossing, T. Mu, M. Murati, O. Murk, D. Mély, A. Nair, R. Nakano, R. Nayak, A. Neelakantan, R. Ngo, H. Noh, L. Ouyang, C. O'Keefe, J. Pachocki, A. Paino, J. Palermo, A. Pantuliano, G. Parascandolo, J. Parish, E. Parparita, A. Passos, M. Pavlov, A. Peng, A. Perelman, F. de Avila Belbute Peres, M. Petrov, H. P. de Oliveira Pinto, Michael, Pokorny, M. Pokrass, V. H. Pong, T. Powell, A. Power, B. Power, E. Proehl, R. Puri, A. Radford, J. Rae, A. Ramesh, C. Raymond, F. Real, K. Rimbach, C. Ross, B. Rotsted, H. Roussez,

- N. Ryder, M. Saltarelli, T. Sanders, S. Santurkar, G. Sastry, H. Schmidt, D. Schnurr, J. Schulman, D. Selsam, K. Sheppard, T. Sherbakov, J. Shieh, S. Shoker, P. Shyam, S. Sidor, E. Sigler, M. Simens, J. Sitkin, K. Slama, I. Sohl, B. Sokolowsky, Y. Song, N. Staudacher, F. P. Such, N. Summers, I. Sutskever, J. Tang, N. Tezak, M. B. Thompson, P. Tillet, A. Tootoonchian, E. Tseng, P. Tuggle, N. Turley, J. Tworek, J. F. C. Uribe, A. Vallone, A. Vijayvergiya, C. Voss, C. Wainwright, J. J. Wang, A. Wang, B. Wang, J. Ward, J. Wei, C. Weinmann, A. Welihinda, P. Welinder, J. Weng, L. Weng, M. Wiethoff, D. Willner, C. Winter, S. Wolrich, H. Wong, L. Workman, S. Wu, J. Wu, M. Wu, K. Xiao, T. Xu, S. Yoo, K. Yu, Q. Yuan, W. Zaremba, R. Zellers, C. Zhang, M. Zhang, S. Zhao, T. Zheng, J. Zhuang, W. Zhuk, and B. Zoph, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [64] G. Park, B. park, M. Kim, S. Lee, J. Kim, B. Kwon, S. J. Kwon, B. Kim, Y. Lee, and D. Lee, "LUT-GEMM: Quantized matrix multiplication based on LUTs for efficient inference in large-scale generative language models," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [65] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research (JMLR)*, vol. 21, no. 140, pp. 1–67, 2020.
- [66] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, "OmniQuant: Omnidirectionally calibrated quantization for large language models," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [67] H. Sharma, J. Park, N. Suda, L. Lai, B. Chau, J. K. Kim, V. Chandra, and H. Esmaeilzadeh, "Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network," in *Proceedings of the ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 764–775.
- [68] M. Shi, V. Jain, A. Joseph, M. Meijer, and M. Verhelst, "Bitwave: Exploiting column-based bit-level sparsity for deep learning acceleration," in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 732–746.
- [69] W. Sun, A. Li, T. Geng, S. Stuijk, and H. Corporaal, "Dissecting tensor cores via microbenchmarks: Latency, throughput and numeric behaviors," *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 34, no. 1, pp. 246–261, 2022.
- [70] G. Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Riviere, M. Kale, J. C. Love, P. D. Tafti, L. Hussenot, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. H'eliou, A. Tacchetti, A. Bulanova, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharmar, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wahltinez, P. Bailey, P. Michel, P. Yotov, P. G. Sessa, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. B. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean, K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin, N. Fiedel, E. Senter, A. Andreev, and K. Kenealy, "Gemma: Open models based on gemini research and technology," *arXiv preprint arXiv:2403.08295*, 2024.
- [71] J. Tian, C. Fang, H. Wang, and Z. Wang, "Bebert: Efficient and robust binary ensemble bert," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.
- [72] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
- [73] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.
- [74] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa, "Quip#: Even better llm quantization with hadamard incoherence and lattice codebooks," *arXiv preprint arXiv:2402.04396*, 2024.
- [75] G. Wang, S. Cai, W. Li, D. Lyu, and G. He, "Bsvit: A bit-serial vision transformer accelerator exploiting dynamic patch and weight bit-group quantization," *IEEE Transactions on Circuits and Systems I: Regular Papers (TCAS-I)*, 2024.
- [76] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "Hsq: Hardware-aware automated quantization with mixed precision," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2019, pp. 8612–8620.
- [77] X. Wei, Y. Zhang, X. Zhang, R. Gong, S. Zhang, Q. Zhang, F. Yu, and X. Liu, "Outlier suppression: Pushing the limit of low-bit transformer language models," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 17402–17414, 2022.
- [78] H. Xia, Z. Zheng, X. Wu, S. Chen, Z. Yao, S. Youn, A. Bakhtiari, M. Wyatt, D. Zhuang, Z. Zhou, O. Ruwase, Y. He, and S. L. Song, "Quant-llm: Accelerating the serving of large language models via fp6-centric algorithm-system co-design on modern gpus," in *2024 USENIX Annual Technical Conference (ATC)*, 2024, pp. 699–713.
- [79] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 38087–38099.
- [80] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [81] Y. Xu, X. Han, Z. Yang, S. Wang, Q. Zhu, Z. Liu, W. Liu, and W. Che, "Onebit: Towards extremely low-bit large language models," *arXiv preprint arXiv:2402.11295*, 2024.
- [82] J. Yuan, H. Liu, S. Zhong, Y.-N. Chuang, S. Li, G. Wang, D. Le, H. Jin, V. Chaudhary, Z. Xu, Z. Liu, and X. Hu, "Kv cache compression, but what must we give in return? a comprehensive benchmark of long context capable approaches," in *The 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [83] Z. Yuan, Y. Shang, Y. Zhou, Z. Dong, Z. Zhou, C. Xue, B. Wu, Z. Li, Q. Gu, Y. J. Lee, Y. Yan, B. Chen, G. Sun, and K. Keutzer, "Llm inference unveiled: Survey and roofline model insights," *arXiv preprint arXiv:2402.16363*, 2024.
- [84] A. H. Zadeh, M. Mahmoud, A. Abdelhadi, and A. Moshovos, "Mokey: Enabling narrow fixed-point inference for out-of-the-box floating-point transformer models," in *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*, 2022, pp. 888–901.
- [85] S. Q. Zhang, B. McDanel, and H. Kung, "Fast: Dnn training under variable precision block floating point with stochastic rounding," in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 846–860.
- [86] S. Zhang, S. Roller, N. Goyal, M. Arretxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "Opt: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.
- [87] Y. Zhang, Y. Du, G. Luo, Y. Zhong, Z. Zhang, S. Liu, and R. Ji, "Cam: Cache merging for memory-efficient llms inference," in *Forty-first International Conference on Machine Learning (ICML)*, 2024.
- [88] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2024.
- [89] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci, "Atom: Low-bit quantization for efficient and accurate llm serving," *Proceedings of Machine Learning and Systems (MLSys)*, vol. 6, pp. 196–209, 2024.